

Compte rendu de TP

Eventify — Plateforme de gestion d'événements

Nom de l'étudiant : Délio Duchesne-Brajon

Groupe : 1

Sommaire

A. Introduction

B. Conception de la solution

C. Réalisation

D. Tests

E. Retour d'expérience

F. Conclusion

Annexes — Listings du code

A. Introduction

Ce TP consistait à développer avec Symfony 7 et MySQL une application web de gestion d'événements, « Eventify », permettant de lister, ajouter, modifier et supprimer des événements (titre, description, date, lieu, nombre de places). Le TP se décomposait en une partie guidée (initialisation du projet, modélisation des données, génération du CRUD, mise en forme du front-end avec Twig et Bootstrap) puis une partie en autonomie portant sur la recherche d'événements, la gestion des inscriptions et la mise en forme des formulaires.

B. Conception de la solution

L'application suit l'architecture MVC imposée par Symfony. Une seule entité, Event, porte les données (titre, description, date, lieu, capacité), mappée en base via Doctrine ORM. Le contrôleur EventController expose les actions de listing, création, consultation et modification, et un repository dédié (EventRepository) centralise les requêtes personnalisées, notamment la recherche par titre.

Deux règles de validation ont été ajoutées sur l'entité : le titre ne peut pas être vide et doit faire au moins 5 caractères, et la capacité doit être un nombre positif.

Pour la recherche, le choix a été de filtrer côté base de données avec une clause LIKE sur le titre, via le QueryBuilder de Doctrine, plutôt que de charger tous les événements et filtrer en PHP. Pour la gestion des inscriptions, la capacité restante est stockée directement comme un entier sur l'événement et décrémentée à chaque inscription, plutôt que de conserver une liste des inscrits.

C. Réalisation

Modèle de données

L'entité Event porte les champs title (string, 255, non vide, 5 caractères minimum), description (text), eventDate (datetime), location (string, 255) et capacity (integer, positif). Une migration Doctrine crée la table correspondante en base MySQL, visible ci-dessous dans phpMyAdmin.



#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
1	id	int(11)			Non	Aucun(e)		AUTO_INCREMENT	Modifier Supprimer Plus
2	title	varchar(255)	utf8mb4_general_ci		Non	Aucun(e)			Modifier Supprimer Plus
3	description	longtext	utf8mb4_general_ci		Non	Aucun(e)			Modifier Supprimer Plus
4	event_date	datetime			Non	Aucun(e)			Modifier Supprimer Plus
5	location	varchar(255)	utf8mb4_general_ci		Non	Aucun(e)			Modifier Supprimer Plus
6	capacity	int(11)			Non	Aucun(e)			Modifier Supprimer Plus

Structure de la table event générée par la migration Doctrine (phpMyAdmin).

Table	Action	Lignes	Type	Interclassement	Taille	Perte
doctrine_migration_versions	Parcourir Structure Rechercher Insérer Supprimer	1	InnoDB	utf8mb4_general_ci	16,0 kio	-
event	Parcourir Structure Rechercher Insérer Supprimer	0	InnoDB	utf8mb4_general_ci	16,0 kio	-
messenger_messages	Parcourir Structure Rechercher Insérer Supprimer	0	InnoDB	utf8mb4_general_ci	32,0 kio	-
3 tables	Somme	1	InnoDB	utf8mb4_general_ci	64,0 kio	0 0

Les 3 tables de la base eventify_db.

Back-end

Le contrôleur et les formulaires ont été générés avec la commande make:crud, puis complétés :

```
PS C:\Users\eleve\eventify> php bin/console make:crud Event

Choose a name for your controller class (e.g. EventController) [EventController]:
> EventController

Do you want to generate PHPUnit tests? [Experimental] (yes/no) [no]:
>
```

Génération du CRUD avec `php bin/console make:crud Event`.

- `index()` : liste les événements, avec un paramètre de requête `q` utilisé pour filtrer par titre via `EventRepository::findBySearch()`.
- `new()` / `edit()` : affichent et traitent le formulaire `EventType` (généré par `make:crud`), avec persistance via Doctrine.
- `show()` : affiche le détail d'un événement et le bouton d'inscription.
- `inscrire()` : route POST qui décrémente la capacité d'un événement de 1 si elle est encore supérieure à 0.

Nom	Modifié le	Type	Taille
.gitignore	03/09/2026 13:05	Fichier source Git I...	0 Ko
EventController.php	03/09/2026 13:52	Fichier source PHP	3 Ko

EventController.php généré dans `src/Controller`.

```

1
2 class EventRepository extends ServiceEntityRepository
3 {
4     public function __construct(ManagerRegistry $registry)
5     {
6         parent::__construct($registry, Event::class);
7     }
8
9     public function findByTitle(?string $title): array
10    {
11        $qb = $this->createQueryBuilder('e');
12
13        if ($title) {
14            $qb->andWhere('e.title LIKE :title')
15                ->setParameter('title', '%' . $title . '%');
16        }
17
18        return $qb->orderBy('e.eventDate', 'ASC')
19            ->getQuery()
20            ->getResult();
21    }
22

```

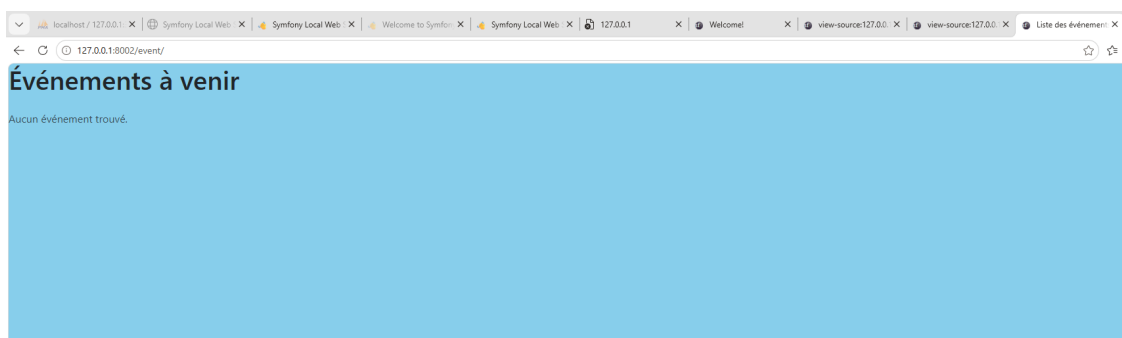
Extrait de EventRepository.php dans VS Code, avec la méthode de recherche findByTitle().

Front-end

Le template de base charge Bootstrap 5 via CDN. La page de liste affiche les événements sous forme de cartes avec un champ de recherche par titre. La page de détail affiche un bouton « S'inscrire » tant qu'il reste des places, remplacé par un badge « Complet » une fois la capacité à 0. Le formulaire de création/modification est mis en forme avec les classes Bootstrap (form-control, form-label, btn-primary).

D. Tests

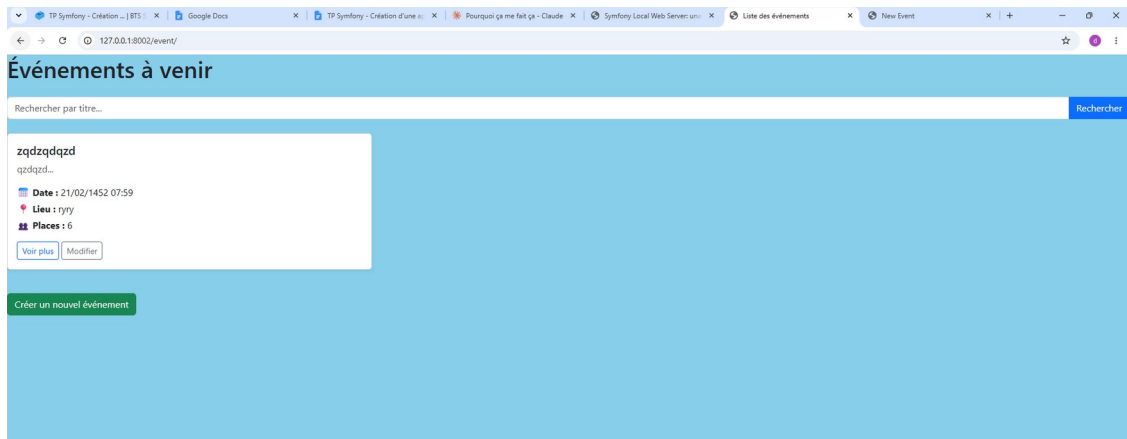
Vérification de la page de liste au fil du développement : d'abord vide juste après la génération du CRUD, puis avec la barre de recherche et le bouton de création une fois la Phase 5 commencée, puis avec un événement réellement créé pour vérifier l'affichage de la carte.



Liste vide juste après la génération du CRUD (aucun événement en base).



Ajout de la barre de recherche et du bouton « Créer un nouvel événement ».



Un événement de test créé et affiché sous forme de carte, avec les boutons Voir plus / Modifier.

E. Conclusion

Le tp était très intéressant mais le plus compliqué c'était l'installation par la suite il suffisait de suivre à la lettre donc ce n'était pas très complexe